

Assignment 2

Oskar Hellebust-Nygaard

Dato: 12. desember 2025

Innhold

1	Formål	2
2	Teori	2
3	Kobling	2
4	Kode	3
5	Resultat	5
6	Konklusjon	5

1 Formål

Målet med denne oppgaven var å lage et enkelt mikrokontrollerprogram som bruker interrupts til å kontrollere to LEDs. Den ene dioden skal blinke hvert femte sekund, mens den andre kan toggles ved hjelp av en knapp koblet til inngangen PD2 (INT0).

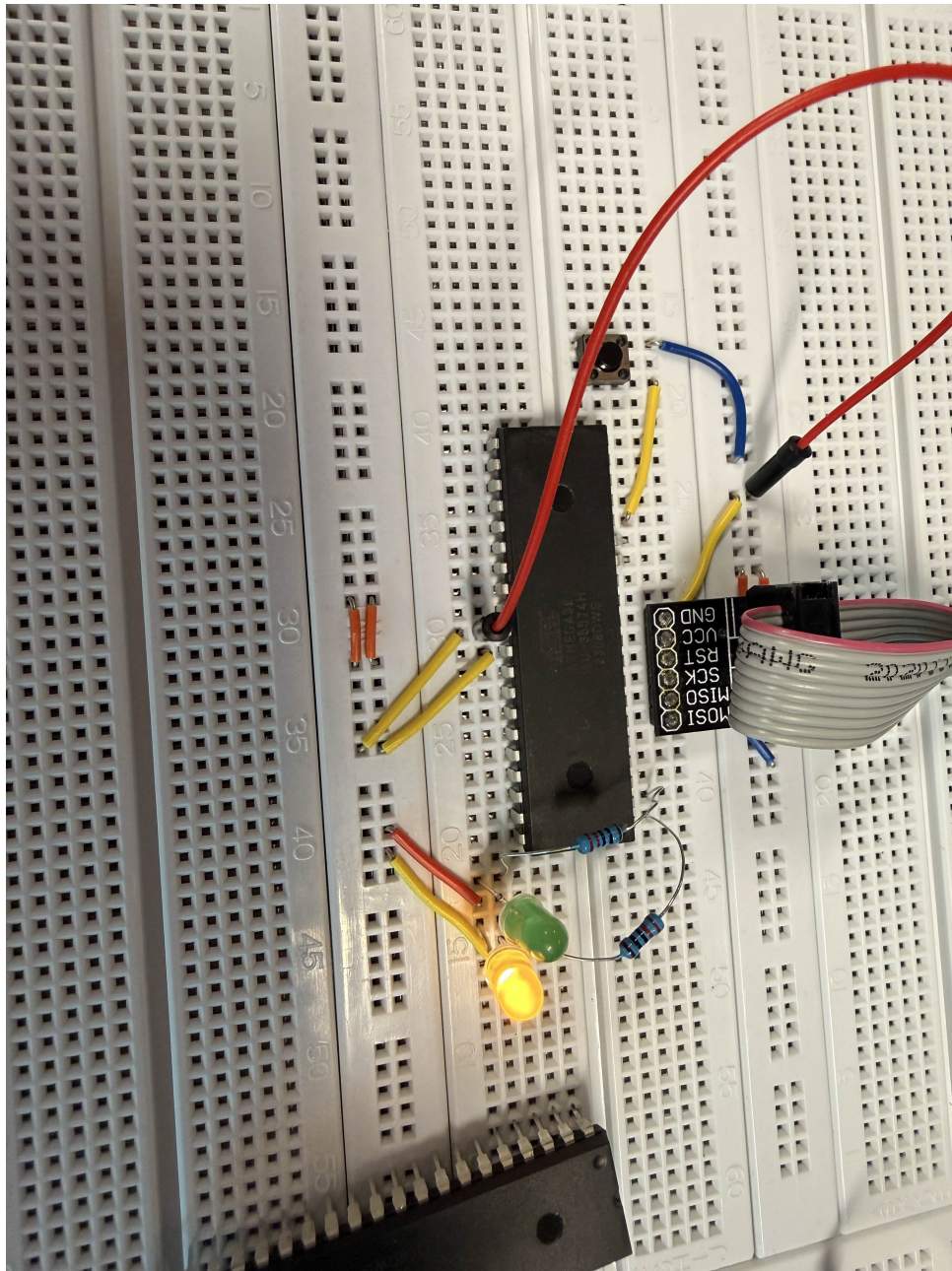
2 Teori

For å løse oppgaven brukes eksterne og interne interruptrs på ATmega32.

- **INT0:** Eksternt interrupts på pin PD2. Konfigureres til å reagere på falling edge når knappen trykkes.
- **Timer1:** Brukes i *CTC-modus* med prescaler /1024 for å generere et 1-sekund-avbrudd. Etter fem slike avbrudd toggles LED 1.
- **Debounce:** Gjennomføres uten bruk av `delay.h`, med en kort tom løkke for å unngå at flere avbrudd registreres ved ett knappetrykk.

3 Kobling

LED-ene er koblet aktiv-HIGH, slik at logisk 1 skruer på lysdioden. Knappen er koblet mellom PD2 og GND, med intern pull-up.



Figur 1: Kobling

4 Kode

Koden under bruker tidsstyrt blinking og knappstyrt togging. Timer1 genererer et 1s delay, og etter fem delay toggles LED 1. Knappens avbrudd INT0 toggler LED 2 etter en enkel debounce.

Listing 1: Programkode for Assignment 2

```
#define F_CPU 1000000UL //1MHz

#include <avr/io.h>
#include <avr/interrupt.h>
#include <stdint.h>
```

```

#define LED_BLINK  PBO    // 5sek
#define LED_BTN    PB1    // toggle med knapp
#define BTN_PIN    PD2    // knapp, int0, pullup

volatile uint8_t sec_count = 0;    // global teller, teller 1s-tick til 5

static inline void io_init(void) {
    // LED AV ved start
    DDRB |= (1<<LED_BLINK) | (1<<LED_BTN);
    PORTB &= ~(1<<LED_BLINK) | (1<<LED_BTN));

    // PD2 input + pull-up
    DDRD &= ~(1<<BTN_PIN);
    PORTD |= (1<<BTN_PIN);
}

static inline void int0_init(void) {
    // INT0 på fallende flanke (trykk -> lav)
    MCUCR |= (1<<ISC01);
    MCUCR &= ~(1<<ISC00);
    GICR |= (1<<INT0);    // enable INT0
}

static inline void timer1_init_1s_tick(void) {
    // Timer1 i CTC for 1 Hz "tick"
    // Bruk prescaler 1024 og sett OCR1A = F_CPU/1024 - 1
    TCCR1A = 0;
    TCCR1B = 0;
    TCCR1B |= (1<<WGM12);    // CTC

    // Beregn 1-sekunds topp
    uint32_t ticks_per_sec = F_CPU / 1024UL;    // 976 ved 1 MHz
    OCR1A = (uint16_t)(ticks_per_sec - 1UL);

    TIMSK |= (1<<OCIE1A);    // enable COMPA interrupt
    TCCR1B |= (1<<CS12) | (1<<CS10);    // start timer, presc = /1024
}

static inline uint8_t debounce_pd2_pressed(void) { //debounce uten delay
    if ((PIND & (1<<BTN_PIN)) == 0) { //if knapp lav
        for (volatile uint16_t i = 0; i < 6000; ++i) { /* tom loop */ } // Mini-vent
        return ((PIND & (1<<BTN_PIN)) == 0); // Sjekk igjen
    }
    return 0;
}

ISR(TIMER1_COMPA_vect) { // 1-sekunds tick, blink hver 5
    if (++sec_count >= 5) {
        PORTB ^= (1<<LED_BLINK);    // toggle PBO
    }
}

```

```

        sec_count = 0;
    }
}

ISR(INT0_vect) { // Knapp på PD2, toggle PB1 med enkel debounce
    GICR &= ~(1<<INT0); // midlertidig disable INT0

    if (debounce_pd2_pressed()) {
        PORTB ^= (1<<LED_BTN);    // toggle PB1
    }

    // clear pending flag og enable igjen
    GIFR |= (1<<INTF0);
    GICR |= (1<<INT0);
}

int main(void) {
    io_init();
    int0_init();
    timer1_init_1s_tick();
    sei();

    for (;;) {
        // alt styres av interrupts
    }
}

```

5 Resultat

Etter kompilering og opplasting på ATmega32 fungerer systemet som forventet:

- LED 1 (PB0) blinker hvert femte sekund automatisk.
- LED 2 (PB1) toggles hver gang knappen trykkes.
- Ingen merkbar “flimmer” eller gjentatte trykk, pga debounce.

6 Konklusjon

Oppgaven viser grunnleggende bruk av **eksterne interrupts** og **tidsavbrudd** på ATmega32 uten bruk av `delay.h`. Resultatet viser hvordan man kan kombinere flere avbrudd for å lage samtidige prosesser i mikrokontroller-programmering.